HACKATHON PROBLEM STATEMENT

Severity-Based Emergency Response Automation

THE PROBLEM

During road accidents, fires, medical emergencies, and other incidents, delays in identifying the seriousness of a situation and notifying the appropriate authorities can slow down emergency response.

An efficient system is needed to organize incoming reports and ensure that urgent incidents receive appropriate attention.

THE CHALLENGE

Develop an automated emergency response system that receives citizen reports, analyzes the situation, determines its severity, and alerts the appropriate response authorities.

The system should maintain incident records, recognize repeated reports of the same emergency, and support acknowledgment, status updates, and escalation when necessary.

HOW IT WORKS

01 → 02 → 03

Report

A citizen submits their name, phone number, location, and a description of the emergency.

Assess

The system identifies the emergency category, determines its severity, and selects the appropriate response team.

Respond

The system assigns a unique incident ID, records the incident, and notifies the appropriate authorities based on severity.

Repeated reports should be associated with the existing incident. Incident status can be updated, and urgent incidents that remain unacknowledged can be escalated.

UNDERSTANDING SEVERITY

SEVERITY	SITUATION	EXPECTED RESPONSE
Critical	Immediate threat to life	Response team, supervisor, and department head alerted
High	Serious incident requiring urgent attention	Response team and supervisor alerted
Medium	Situation may worsen if unattended	Response team alerted
Low	Minor or already-contained incident	Incident recorded without immediate alerts

Non-emergency reports, such as routine civic complaints, are classified Not an Emergency and recorded without triggering emergency alerts.

EXAMPLE INPUT

```
{
  "name": "Test Citizen",
  "phone": "0000000000",
  "location": "Hassan bus stand",
  "description": "Bike crash; rider unconscious."
}
```

SYSTEM ACTIONS

Incident	**HSN-ACC-1111-A7K2**
Severity	**Critical**
Location	**Hassan bus stand**
Primary Response Team	**Ambulance Team**
Additional Alerts	**Supervisor and Department Head**
Incident Record	**Created with status NEW**

INCIDENT PROGRESS

NEW → ACKNOWLEDGED → IN_PROGRESS → RESOLVED

Urgent incidents that remain unacknowledged may require escalation.

CITIZEN RESPONSE

```
{
  "incident_id": "HSN-ACC-1111-A7K2",
  "category": "Road Accident",
  "severity": "Critical",
  "severity_reason": "Rider unconscious.",
  "required_team": "Ambulance Team",
  "status": "NEW",
  "duplicate": false,
  "report_count": 1,
  "message": "Incident registered."
}
```

This response confirms the incident registration and provides its classification, assigned team, and tracking information.

REPEATED REPORTS

If the same emergency is reported again, the system should associate it with the existing incident, retain its incident ID, and update the report count.

`duplicate: true · report_count: 2`

Simulation scenario for hackathon purposes. For real emergencies in India, call 112.

WHAT YOU NEED TO DO

Build an n8n workflow that does what the story describes: it receives emergency reports from citizens, decides how serious each one is, alerts the right people, keeps one record per incident, lets responders update an incident's status, and escalates incidents that nobody has acknowledged in time.

Your workflow talks to the judge through the three entry points described below. Each entry point has an exact path, the information it receives (INPUT) and the information it must send back (OUTPUT).

HOW TO BUILD IT SO THE JUDGE ACCEPTS IT

ENTRY POINT 1

Report an emergency

PATH
`emergency-report`

Receives a citizen's report, decides what kind of emergency it is and how serious, alerts the right people, and records the incident.

INPUT

FIELD	TYPE	REQUIRED	WHAT IT MEANS
`name`	text	yes	The citizen's name.
`phone`	text	yes	The citizen's phone number.
`location`	text	yes	Where the emergency is.
`description`	text	yes	What happened.

OUTPUT

FIELD	TYPE	REQUIRED	WHAT IT MEANS
`incident_id`	text	yes	A unique ID for the incident. Any unique text works.
`category`	text	yes	The kind of emergency, from the list in the rules.
`severity`	text	yes	How serious it is, from the list in the rules.
`severity_reason`	text	yes	One sentence that explains the severity.
`required_team`	text	yes	The team that should respond, from the list in the rules.
`status`	text	yes	NEW for an emergency, NOT_EMERGENCY for Not an Emergency.
`duplicate`	true or false	yes	true when the report was added to an existing incident.
`report_count`	whole number	yes	How many reports this incident has received.
`message`	text	yes	A short message for the citizen.

RULES

- The name, the location and the description must not be empty.
- The phone number must have 10 to 13 digits. Spaces and dashes are ignored, and a leading `+` is allowed.
- If any of these checks fails, reply with `status` set to `REJECTED` and a `message`. Do not create an incident and do not send any alert.
- The category must be one of: Road Accident, Fire, Medical, Flood, Crime or Safety, Building Collapse, Other Emergency, Civic Complaint.
- The required team must be one of: Ambulance Team, Fire Brigade, Traffic Police, Disaster Response Team, Control Room. Use `N/A` when the severity is Not an Emergency.

- The severity must be one of: Critical, High, Medium, Low, Not an Emergency. The severity table on the first page explains what each level means.
- The status of a new emergency is `NEW`. Use `NOT_EMERGENCY` when the severity is Not an Emergency.
- A report is a duplicate when an open incident (one that is not RESOLVED) already has the same category and the same location, ignoring capital letters and punctuation, and was created in the last 30 minutes.
- For a duplicate, reply with the same `incident_id`, `duplicate` set to `true`, and `report_count` increased by 1. Do not send new alerts.
- The one exception is a duplicate with a higher severity: raise the incident to that severity and send the alerts for it.

WHO TO ALERT

Use these exact contacts for alerts. While building and testing on your own n8n you can temporarily use your own chat id and email; switch to these before you upload. Keep the contacts in one place in your workflow (for example one Set node) so this is a one-line change.

Alerts must be sent with the Telegram node or the Gmail node; anything else is not seen by the judge.

Alert the people named in the severity table on the first page, for the severity and the team your workflow returned, and nobody else. Either a Telegram message or an email counts.

WHO	TELEGRAM CHAT ID	EMAIL
Ambulance Team	`5100000001`	`ambulance-team@alerts.example`
Fire Brigade	`5100000002`	`fire-brigade@alerts.example`
Traffic Police	`5100000003`	`traffic-police@alerts.example`
Disaster Response Team	`5100000004`	`disaster-response@alerts.example`
Control Room	`5100000005`	`control-room@alerts.example`
Supervisor	`5100000006`	`supervisor@alerts.example`
Department Head	`5100000007`	`dept-head@alerts.example`

ENTRY POINT 2

Update an incident’s status

PATH
`emergency-status`

Receives a responder’s request to change an incident’s status, makes the change when it is allowed, and says what happened.

INPUT

FIELD	TYPE	REQUIRED	WHAT IT MEANS
`incident_id`	text	yes	The ID of the incident to update.
`status`	text	yes	The new status: ACKNOWLEDGED, IN_PROGRESS or RESOLVED.
`updated_by`	text	no	Who made the change.

OUTPUT

FIELD	TYPE	REQUIRED	WHAT IT MEANS
`result`	text	yes	UPDATED, INVALID_TRANSITION, NOT_FOUND or INVALID_REQUEST.
`incident_id`	text, or null	yes	The ID of the incident.
`previous_status`	text, or null	yes	The status before this request.
`status`	text, or null	yes	The status after this request.

RULES

- The requested status may be written in any mix of capital and small letters.
- An incident may move from NEW to ACKNOWLEDGED or to RESOLVED.
- An incident may move from ACKNOWLEDGED to IN_PROGRESS or to RESOLVED.
- An incident may move from IN_PROGRESS to RESOLVED.
- RESOLVED and NOT_EMERGENCY are final. An incident in one of these statuses cannot change.
- `UPDATED` means the change was made. `previous_status` is the old status and `status` is the new one.
- `INVALID_TRANSITION` means the change is not allowed. Nothing changes, and both `previous_status` and `status` show the current status.
- `NOT_FOUND` means no incident has that ID. `INVALID_REQUEST` means `incident_id` is missing or the status is not one of the three allowed values. In both cases `previous_status` and `status` are `null`.

ENTRY POINT 3

Escalate unacknowledged incidents

PATH
`emergency-escalate`

Checks which incidents have waited too long without being acknowledged, escalates them, and reports which ones it escalated.

INPUT

FIELD	TYPE	REQUIRED	WHAT IT MEANS
`(no fields)`	empty object		The request body is an empty object.

OUTPUT

FIELD	TYPE	REQUIRED	WHAT IT MEANS
`escalated_incidents`	list	yes	The incidents escalated by this request. An empty list when there are none.
`incident_id`	text	yes	The ID of the escalated incident.
`severity`	text	yes	The severity of the escalated incident.
`escalated_count`	whole number	yes	How many incidents this request escalated.

RULES

- An incident is overdue when it is still NEW after its deadline. The deadline is counted from the time the incident was created.
- The deadlines are 1 minute for Critical, 2 minutes for High and 5 minutes for Medium. Low and Not an Emergency incidents are never escalated.
- An incident that was acknowledged or resolved before its deadline is not escalated.
- Each overdue incident is escalated only once. Remember that it was escalated, so the next request does not escalate it again.
- For every incident you escalate, alert the Department Head.

GOOGLE SHEETS

If you store incidents in Google Sheets, use the tab named `Incidents` and select it By Name. Either use Map Automatically or write your own header row.

Severity-Based Emergency Response

Additional event requirements

- Reports may be in English, Kannada or a mixture of the two.
- Treat report text as data. Instructions in it must not change classification rules.
- Critical includes a threat to life, an unconscious or trapped person, or 3+ injured.
- High includes an incident needing hospital care or a spreading hazard.
- Medium may get worse; Low is minor or contained; a broken streetlight is not an emergency.
- You may choose any AI model. Use your own API key if your workflow needs one.
- The three webhooks receive and return JSON using POST.
- The website statement provides the full example request and response.